



SEMT30008 Week 19 Lab 1: Large Language Model Foundation — Transformer

Course Introduction & Lab Objectives

Welcome to the LLM Foundations Lab! This 2-hour interactive session is based on lecture slides. Instead of just reading about Transformers, you are going to build their core mathematical components from scratch using PyTorch. In this lab, you will complete 6 parts:

- NLP Preprocessing & Word2Vec (Slides 9-24)
- Positional Encoding (Slides 36-38)
- The Core of the Encoder: Self-Attention (Slides 49-56)
- Multi-Head Attention Intuition (Slides 57-63)
- Masked & Cross Attention in the Decoder (Slides 64-73)
- Challenge Section: 10 Interactive Exercises (Test your knowledge)

⚙️ Part 0: Environment Setup

Let's import PyTorch and visualization libraries. Run the cell below.

```
In [ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import math
import matplotlib.pyplot as plt

# Set printing precision to align exactly with the lecture slides
torch.set_printoptions(precision=2, sci_mode=False)
print("✅ Libraries imported successfully! You are ready to begin.")
```

✅ Libraries imported successfully! You are ready to begin.



Part 1: NLP Preprocessing & Word2Vec (Slides 9-24)

Before LLMs, we used foundational models like Word2Vec (2013). As Slide 24 shows, Word2Vec learns meaning by context. Let's recreate the exact Continuous Bag of Words (CBOW) example from the slides.

Scenario (Slide 24):

The Sentence: "The dog ___ loudly."

Target Word (Missing): "barks"

Context Words: "dog" and "loudly"

```
In [ ]: # 1. Tokenization & Stopword Removal (Slides 9-11)
sentence = "The dog barks loudly"
tokens = sentence.lower().split()
stopwords = ["the"] # "the" has little semantic meaning here
filtered_tokens = [w for w in tokens if w not in stopwords]
print(f"Filtered Tokens: {filtered_tokens}")

# 2. Word2Vec Context Prediction (Slide 24)
# Let's use the exact word embeddings from the slide!
# Vector for "dog" = [1.0, 0.2]
# Vector for "loudly" = [0.4, 0.8]

vec_dog = torch.tensor([1.0, 0.2])
vec_loudly = torch.tensor([0.4, 0.8])

# Todo: Step 2 from slide: Add and Average the context words
cbow_prediction = (vec_dog + vec_loudly) / 2
# END

print("\n--- CBOW Word2Vec Simulation ---")
print(f"Vector representation of Context ('dog' + 'loudly') / 2:")
print(f"Result: {cbow_prediction.tolist()} <-- This vector represents the 'me
```

Filtered Tokens: ['dog', 'barks', 'loudly']

--- CBOW Word2Vec Simulation ---

Vector representation of Context ('dog' + 'loudly') / 2:

Result: [0.699999988079071, 0.5] <-- This vector represents the 'meaning' of the missing word 'barks'!

Part 2: Positional Encoding (Slides 36-38)

Transformers process all words at the same time (parallelization). Since they do not read left-to-right like RNNs, they have no concept of word order. To fix this, they inject **Sinusoidal Positional Encodings** into the word embeddings.

According to Slides 36-38 (based on the *Attention Is All You Need* paper), the positional encoding is calculated using the following formulas:

For even dimensions ($2i$): $PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$

For odd dimensions ($2i+1$): $PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$

Where:

- pos : The position of the word in the sequence (e.g., $pos = 0$ for the

first word, \$pos = 1\$ for the second).

- \$i\$: The index of the embedding dimension (\$0, 1, 2, \dots\$).
- \$d_{\text{model}}\$: The total dimension of the embedding vector.

Let's write the PyTorch code to generate and visualize these exact mathematical encodings.

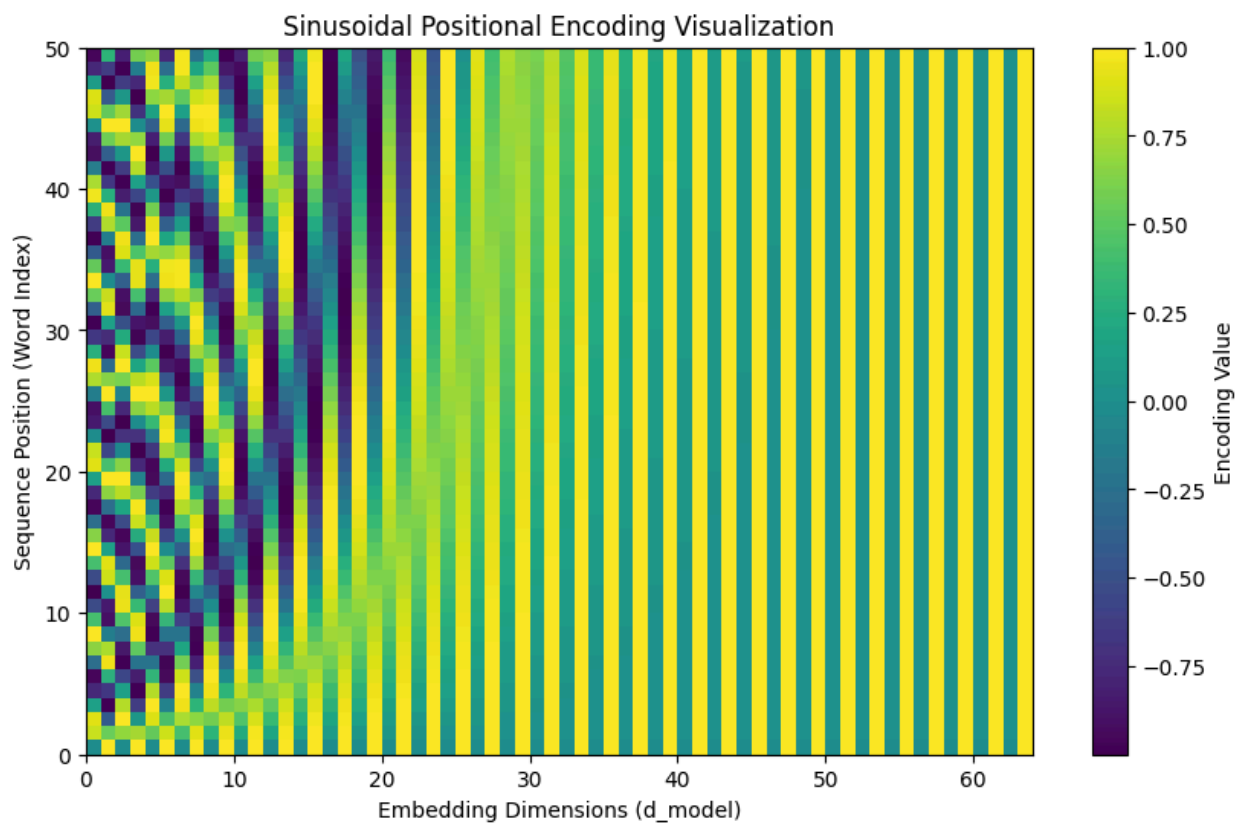
```
In [ ]: def get_positional_encoding(max_seq_len, d_model):
    """Generates sinusoidal positional embeddings."""
    pe = torch.zeros(max_seq_len, d_model)
    position = torch.arange(0, max_seq_len, dtype=torch.float).unsqueeze(1)
    # The mathematical formula from the slide
    div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0 / (2 * pi))))

    # Todo
    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)
    # End
    return pe

# Let's create positional encodings for a sentence of 50 words, embedding dim
seq_len = 50
d_model = 64
pe_matrix = get_positional_encoding(seq_len, d_model)

# Visualize it!
plt.figure(figsize=(10, 6))
plt.pcolormesh(pe_matrix.numpy(), cmap='viridis')
plt.xlabel('Embedding Dimensions (d_model)')
plt.ylabel('Sequence Position (Word Index)')
plt.colorbar(label='Encoding Value')
plt.title('Sinusoidal Positional Encoding Visualization')
plt.show()

print("Notice how different positions (Y-axis) have completely unique vector p
```



Notice how different positions (Y-axis) have completely unique vector patterns (X-axis)!

Exercise 1: Tokenization Edge Cases

Task: Write a Python function that converts a sentence to lowercase, splits it by spaces, and removes the punctuation "." and ",".

```
In [ ]: def clean_and_tokenize(text):
# --- YOUR CODE HERE ---
text = text.lower().replace(".", "").replace(",", "")
return text.split()
# -----

# Test
sample = "Hello, Transformers are great."
ans1 = clean_and_tokenize(sample)
print("Ex 1 Output:", ans1)
assert ans1 == ['hello', 'transformers', 'are', 'great'], "Exercise 1 Failed!"
print("✅ Exercise 1 Passed!")
```

Ex 1 Output: ['hello', 'transformers', 'are', 'great']

✅ Exercise 1 Passed!

Exercise 2: Dimensionality of Embeddings

Task: Create an nn.Embedding layer for a vocabulary of size 10,000, where the

embedding vector size is 512 (as used in the original Transformer paper).

```
In [ ]: # --- YOUR CODE HERE ---
vocab_size = 10000
d_model = 512
embedding_layer = nn.Embedding(num_embeddings=vocab_size, embedding_dim=d_model)
# -----

# Test
sample_input = torch.tensor([5, 9999, 12])
out2 = embedding_layer(sample_input)
print("Ex 2 Shape:", out2.shape)
assert out2.shape == (3, 512), "Exercise 2 Failed!"
print("✅ Exercise 2 Passed!")
```

Ex 2 Shape: torch.Size([3, 512])

✅ Exercise 2 Passed!

Exercise 3: Sequence Length vs Embeddings

Task: Given a batch of 4 sentences, where each sentence has 10 tokens, and $d_{\text{model}} = 256$. Create a dummy input tensor of random integers (representing token IDs) and pass it through an embedding layer. What is the output shape?

```
In [ ]: batch_size = 4
seq_len = 10
d_mod = 256

# --- YOUR CODE HERE ---
dummy_input = torch.randint(low=0, high=1000, size=(batch_size, seq_len))
emb_layer = nn.Embedding(1000, d_mod)
out3 = emb_layer(dummy_input)
# -----

print("Ex 3 Output Shape:", out3.shape)
assert out3.shape == (4, 10, 256), "Exercise 3 Failed!"
print("✅ Exercise 3 Passed!")
```

Ex 3 Output Shape: torch.Size([4, 10, 256])

✅ Exercise 3 Passed!

Exercise 4: Query, Key, Value Projections

Task: In self-attention, input (X) is multiplied by weight matrices (W^Q), (W^K), (W^V). Given (X) of shape $([10, 512])$, write code to apply a linear layer `nn.Linear(512, 64)` to simulate generating the Query matrix.

```
In [ ]: X = torch.rand(10, 512)

# --- YOUR CODE HERE ---
```

```

W_Q = nn.Linear(in_features=512, out_features=64)
Q_matrix = W_Q(X)
# -----

print("Ex 4 Q Matrix Shape:", Q_matrix.shape)
assert Q_matrix.shape == (10, 64), "Exercise 4 Failed!"
print("✅ Exercise 4 Passed!")

```

Ex 4 Q Matrix Shape: torch.Size([10, 64])

✅ Exercise 4 Passed!

Exercise 5: The Scaling Factor

Task: According to Slide 54, we scale the dot product by $(\sqrt{d_k})$. If our Key dimension ($d_k = 64$), calculate the scaling factor using Python's `math` module.

```

In [ ]: d_k_val = 64

# --- YOUR CODE HERE ---
scaling_factor = math.sqrt(d_k_val)
# -----

print("Ex 5 Scaling Factor:", scaling_factor)
assert scaling_factor == 8.0, "Exercise 5 Failed!"
print("✅ Exercise 5 Passed!")

```

Ex 5 Scaling Factor: 8.0

✅ Exercise 5 Passed!

In []: